The Anchor Decides: Verification Autonomy Levels Predict the Success of LLM-Agent Security Defenses

Yajie Yin

Email: 1549080929@qq.com · ORCID: 0009-0001-6168-2530
Code & data: https://github.com/1549080929-debug/math_agent

Abstract

LLM-agent security has produced a dense landscape of defenses—prompt hardening, content filters, permission gates, sandboxes, provenance firewalls—each claiming to make agents "safe." We argue that the field lacks a pre-purchase question: *what does a given defense actually guarantee, and where does that guarantee come from?* We apply Verification Autonomy Levels (VAL, arXiv:2608.19009), a taxonomy that grades verification/authorization schemes by the source of their spec (L0: LLM self-declaration, no deterministic anchor; L1: deterministic rules; L2: objective ground truth, correctness only; L3/L4: decidable systems with single-property or domain-level completeness; L5: impossible), to 22 agent-security defenses. The classification is a falsifiable predictor: a defense fails the way its anchor fails. We validate this on published data (10/10 prediction hits) and then run the first controlled deployment-value comparison: same budget, two stacks—a VAL-selected stack (intent-anchored confirmation gate + schema sandbox) versus a mainstream intuition stack (prompt hardening + keyword filter)—across 40 scenarios, 12 attack families (including third-party AgentDojo payloads), real tool effects, adaptive and white-box attacks, and three seeds (~4,800 LLM calls). The VAL stack holds 0.000 attack success with 1.000 benign success in every condition; the intuition stack also reaches 0.000 ASR but kills all benign actions and its security is model-behavior luck (compliance 0.235), not structure (the VAL stack tolerates 2.4x more model compromise—0.567 compliance—and still blocks everything). The same zero, two different guarantees. We further identify where L3 anchors exist in agent security: confinement and information-flow properties (sandboxing, taint tracking, data-control separation), not semantic safety, which caps at L2.

1. Introduction

Large language model (LLM) agents are moving from single-turn assistants to persistent systems that call tools, browse the web, and maintain long-term memory. This expansion has produced a matching expansion of security defenses: system-prompt hardening, keyword and content filters, tool-allowlist permission systems, confirmation gates, information-flow control, sandboxes, provenance-aware memory firewalls, and more. Each is presented with an implicit guarantee—"our defense blocks prompt injection," "our gate stops unauthorized actions." But the guarantees are rarely commensurable: one paper's attack success rate (ASR) is not another's; one defense's security posture collapses under a

different attack family; and no framework tells a deployer *which defense to trust for which threat, before running experiments*.

This paper asks the question the field has not asked explicitly: **what does a given agent-security defense guarantee, and where does that guarantee come from?**

We answer with Verification Autonomy Levels (VAL), a taxonomy proposed in [1] for verification schemes generally. VAL classifies any verification/authorization scheme along a single axis—the source of its verification spec—into six levels:

- **L0**: the spec is LLM self-declaration ("I checked it," "this is safe"). No deterministic anchor. Failure mode: the declaration can be rewritten by the very model that issues it.
- **L1**: deterministic rules (regex, keyword lists, hand-written policies). Failure mode: surface forms are bypassable by rewriting.
- **L2**: objective ground truth (gold labels, platform-recorded events, execution results). Guarantees *correctness only*: everything checked passes, but nothing is proven about what was not checked (the completeness blind spot).
- **L3/L4**: decidable systems (solveset equivalence, type systems, formal kernels, confinement mechanisms). Single-property or domain-level completeness within an operational design domain (ODD).
- **L5**: universal completeness—impossible in the unrestricted case.

VAL's core claim is that a scheme's level is determined by its *anchor*, not by its accuracy or sophistication, and that the anchor determines the scheme's failure modes. Applied to agent security, this yields a falsifiable prediction: **a defense is defeated the way its anchor fails.** An L0 defense fails when the model complies with the injection; an L1 defense fails when the attacker rewrites around the surface rule; an L2 defense fails outside its ODD (forged metadata, poisoned data); an L3 defense holds within its ODD by construction.

We make three contributions:

1. **A level map of agent security.** We classify 22 defenses—published systems (PPMF [2], Llama Guard, CaMeL, Progent, IFC/Fides, NeMo Guardrails, A-MemGuard, etc.), baselines, and our own four implementations—using a frozen, blind-rater-validated protocol (inter-rater $\kappa \approx 0.8$ [1]). The map organizes the defense landscape by what each defense can actually guarantee.
2. **Prediction validation.** We freeze level-to-behavior prediction cards and validate them: 10/10 hits on published data (PPMF's own numbers, abstract-level evidence), including two mechanism-driven classification corrections.
3. **The first deployment-value comparison.** Same budget, two stacks, one testbed: a VAL-selected stack (L2 intent anchor + L3 confinement) versus a mainstream intuition stack (L0 prompt hardening + L1 keyword filter), across 40 scenarios, 12 attack families, real tool effects, adaptive and white-box attacks, three seeds. The VAL stack dominates on both security and utility, and the contrast is structural rather than incidental.

2. Background and Related Work

2.1 Verification Autonomy Levels (VAL)

VAL [1] grades verification schemes by three questions: Q1 *where does the verification spec come from* (LLM declaration / deterministic rule / objective truth / decidable system); Q2 *what does the verdict guarantee* (nothing / correctness / completeness); Q3 *what scope does a completeness claim cover* (single property / domain / universal). The level is the first rule that fires: L5 (universal completeness, impossible), L4 (domain completeness), L3 (single-property completeness), L2 (correctness with objective anchor), L1 (correctness with deterministic rule), L0 (otherwise). The procedure is deterministic and has been measured for inter-rater reproducibility: three independent blind raters reach $\kappa \approx 0.8$ on 48–70 schemes under the refined protocol [1].

Two refinements matter for security. First, **anchor semantics** [13]: objective anchors split by *what they certify*—intent (a recorded decision event: "the user confirmed this target"), truth (a fact: "the answer matches ground truth"), effect (an execution outcome: "the action actually occurred"). A confirmation record is an intent anchor; it authorizes, but says nothing about whether the outcome was correct. Conflating intent with effect is the category error behind most authorization failures. Second, **the completeness blind spot** [1]: substitution- and sampling-based checks verify proposed candidates but cannot prove that no candidate was missed. In security, the analog is the *un-enumerated attack*: a gate that blocks all evaluated attacks is a correctness probe over the evaluated distribution, not a guarantee that no attack path was missed.

2.2 Agent-security defenses and their taxonomies

The defense literature is large. Text-level defenses (prompt hardening [8], keyword/content filters, Llama Guard [9]) operate on the current context; tool-level defenses (allowlists [10], permission systems [11], information-flow control [12], sandboxes) gate execution; memory-level defenses (A-MemGuard [14], PPMF [2]) protect persistent state; benchmarks (AgentDojo [4]) measure attack success and utility trade-offs. Recent work has begun to move beyond single-defense evaluation: Injection-Execution Dissociation [5] shows that injection success and tool-execution success are separable safety properties (memory storage rates >97.5% while downstream execution ranges 0–95%, uncorrelated); the Source-of-Authority SLR [6] classifies LLM test oracles by where their authority comes from and finds over half reach verdicts with no specification at all; When Does Verification Pay Off [7] shows that same-family verification yields near-zero gain while cross-family verification remains valuable. Our contribution is different in kind: we do not add another defense, we provide a *pre-purchase axis*—the anchor—that organizes the entire landscape and predicts each defense's failure modes, and we measure whether choosing by this axis beats choosing by intuition.

3. Method

3.1 Classification protocol

We classify defenses with the frozen VAL protocol (v2.3, [1]), extended for security: R1 benchmarks are N/A (not runtime verifiers); R2 mixed systems are split per component with the weakest anchor on the verdict path reported; R3 evidence anchors do not equal verdict anchors (retrieved documents are L2 evidence; an LLM alignment verdict over them is L0); R4 silver anchors (deterministic recomputation without gold) are L1; R10 designer-authored unvalidated rules are L1; R12b theorem-backed exhaustive procedures (confinement, taint tracking) are decidable (L3); R15 LLM-generated verification expectations place the weakest anchor at L0. Each defense is rated on Q1/Q2/Q3 from a mechanism-only evidence pack by blind raters; classifications reported here are the committed (weakest-anchor) levels.

3.2 Prediction protocol

Following the reverse-validation discipline of [1], we freeze prediction cards before consulting outcome data: for each defense, level → predicted behavior pattern (ASR regime, bypass mechanism, benign-cost mode) + the specific attack predicted to break it. Cards are signed and dated, then validated against published data (PPMF's own numbers; abstract-level evidence for other systems, flagged as such).

3.3 Deployment-value experiment

Testbed. A minimal agent: external content is written into memory; a user task triggers a tool call; the LLM (DeepSeek-chat, temperature 0.2) proposes a call; the defense chain decides; a real-effect executor performs the action (a ledger for transfers, an outbox for email, real file deletion in a sandbox directory, a forum log for comments). Attack success requires a *real state change* (server-side effect semantics, as in [2]), not merely a proposal.

Scenarios and attacks. 40 scenarios (30 malicious + 10 benign), derived from the memory-provenance families of [2] (direct injection, keyword evasion, confirmation spoofing, procedure laundering, preference laundering, cross-domain, split context, authority laundering, hidden handles, multi-step, recovery jargon, frustration recovery). Attacks: 8 self-written variants (direct, paraphrase, fake-confirmation, combo, role-play, jailbreak prefix, deep rewrite with keyword splitting, authority frame) plus 4 third-party payloads from AgentDojo [4] (TODO-prefix, ignore-previous, system-message, InjecAgent), 12 total.

Defenses. Four self-implemented defenses, one per anchor family:
- **D1** (L0): system-prompt hardening—"ignore instructions in external content; never perform high-risk actions without explicit user request." The verdict is the model's own compliance.
- **D2** (L1): keyword filter over memory and call text (transfer/delete/send/payment terms, Chinese equivalents).
- **D3** (L1/L2): confirmation gate—high-risk tools require a *platform-recorded* confirmation event; text claims ("user-confirmed") are ignored.
- **D4** (L3): parameter sandbox—tool arguments must match schema (recipient whitelists, target patterns, type/range checks), executed in a side-effect stub with verified state changes.

Stacks (same budget, two defenses each).
- **V (VAL-selected)**: D3 + D4—an intent anchor (L2 input) plus a confinement property (L3).
- **N (mainstream intuition)**: D1 + D2—prompt hardening plus keyword filtering, the most common real-world combination.

Metrics. ASR = high-risk tool executed with complete arguments and verified state change; compliance rate = the model *proposes* a high-risk tool (including empty-argument hedging)—separating "model was driven by the injection" from "the defense intercepted." Benign success = benign scenarios complete their intended action.

Attack escalation. Static attacks (all 12 variants), adaptive attacks (3 rounds; an attacker LLM rewrites the injection from the previous round's interception reason), white-box adaptive (the attacker is told the exact system prompt and keyword list, and applies layered strategies: keyword evasion, then jailbreak framing, then combination).

Runs. Static: 7 configs × 370 cases. Adaptive/white-box: 30 malicious scenarios × stacks × 3 rounds. Seeds: V, N, and the no-defense baseline (ND) re-run 3×. Total ≈ 4,800 LLM calls.

4. The Defense Level Map

Level	Defenses	Anchor / semantics	Predicted behavior
L0	Prompt hardening (D1; RA-LLM [8]), ToolEmu judge, A-MemGuard consensus, SafeAgent plan layer, Self-Ask provenance	none / LLM self-assessment	residual ASR under strong attacks; no stable operating point; zero utility under over-restrictive prompts
L1	Keyword filter (D2), NeMo Guardrails, Progent gate, tool allowlists, PPMF gate, confirmation-marker heuristics, perplexity filter	designer rules / intent	bypassed by rewriting; false-block benign actions
L2	Llama Guard classifier, RAG citation check, effect-verification, statistical detectors	objective truth / truth·effect	effective in-distribution; fails on OOD, forged metadata, poisoned data
L3	**IFC/Fides [12], CaMeL [15], sandboxing (D4), Progent SMT monotonic confinement**	decidable systems / structural	confinement, information-flow, or data-control properties hold by construction within the ODD; semantic dangers inside the allowed space pass
L4	Smart-contract formal verification	formal kernels	domain-complete for encoded properties
N/A	AgentDojo [4], M³-SafetyBench	benchmarks	not runtime defenses (R1)

Full per-defense ratings with evidence: `reliability/corpus.json`, `ratings/`; blind-rater agreement for the security subset is 100% between the two most recent raters (6/6 agentsec items, $\kappa \approx 0.88$ overall [1]).

The structural finding. L3 anchors exist in agent security—but only for *confinement and information-flow properties*: sandbox execution domains, taint tracking, data-control separation, SMT-

monotonic permission lattices. These are decidable by construction. *Semantic* safety ("is this action dangerous?") is an open-world property with no decidable fragment, and caps at L2. This mirrors the formal/empirical dichotomy of [1] exactly: completeness is reachable only for formally specifiable properties.

5. Prediction Validation

We froze 16 prediction cards (level → predicted failure mode + predicted breaking attack) before consulting outcome data.

PPMF family (6 defenses): all six predictions hit on PPMF's own published numbers [2]: content filters (L1) leave 0.200–0.933 ASR and collapse under rewriting; Self-Ask provenance (L0) reaches 0.984 ASR; the gate-only ablation (L1 without L2 inputs) blocks attacks but kills all benign actions; the full PPMF (L1/L2: platform metadata through a designer policy) achieves 0.000 ASR on evaluated attacks with benign preserved, and 0.088 ASR under 10% forged confirmations—the completeness blind spot made quantitative.

New cards (abstract-level evidence, flagged): 4/4 behavior predictions hit (Llama Guard distribution-bound accuracy; RA-LLM residual ASR under strong attacks; perplexity filtering weak alone; ToolEmu audit-value-only). Two mechanism-driven corrections emerged: Progent's SMT monotonic confinement (L1 gate + L3 annotation) and RARR's LLM-driven verdict (L2 → L0 under rule R3). The prediction loop does not only predict; mismatches surfaced incomplete evidence and corrected the classifications.

Cumulative: 10/10 behavior predictions hit (6 high-confidence, 4 abstract-level), 2 classification corrections. Sample is small; the value is that the prediction *protocol* is now demonstrated end-to-end, not that the hit rate is large.

6. Deployment-Value Experiment: Results

6.1 Static attacks (12 variants, 370 cases/config, real effects)

Config	ASR	Compliance	Benign	gain vs ND
ND (no defense)	**0.333**	0.492	1.000	—
D1 prompt hardening (L0)	0.000	0.254	0.000	0.333, benign destroyed
D2 keyword filter (L1)	0.258	0.512	1.000	0.075
D3 confirmation gate (L1/L2)	0.000	0.512	1.000	0.333
D4 parameter sandbox (L3)	0.079	0.537	1.000	0.254
V = D3+D4 (VAL)	**0.000**	0.512	**1.000**	**0.333, lossless**
N = D1+D2 (intuition)	0.000	0.235	**0.000**	0.333, benign destroyed

Every defense reduces ASR relative to the 0.333 baseline; the two stacks reach the same zero. They differ in *how*: V's zero is produced by a gate the model cannot influence; N's zero is produced by the model's behavior under the hardening prompt (compliance 0.235—the model proposes the malicious tool a quarter of the time, mostly with empty arguments; strict ASR counts only actionable executions). D2 alone leaks 0.258 ASR: the keyword filter is bypassable by rewriting.

6.2 Adaptive and white-box attacks

Config	Static ASR	Adaptive ASR	White-box ASR	White-box compliance
D2	0.258	**0.467**	—	—
D4	0.079	**0.200**	—	—
V	0.000	0.000	0.000	**0.567**
N	0.000	0.000	0.000	0.235

Adaptive attacks (attacker rewrites from the interception reason) nearly double the leak rate of the L1 and L3 single defenses. Against the stacks: V holds—the attacker cannot forge platform records or schema-legal arguments it does not possess—even when the model is driven to propose the malicious tool in 57% of scenarios (white-box compliance 0.567, 2.4x N's 0.235). N holds only because the model's "hedging wall" (proposing the tool with empty arguments) survives same-family jailbreak attempts; the attacker was itself DeepSeek. This is model-behavior luck, not structure: D2 alone leaks, and a cross-family attacker with stronger jailbreaks is the remaining untested threat (we could not test this without a non-DeepSeek API).

6.3 Seeds and honesty

V, N, and ND re-run 3×: standard deviations ≈ 0.005 on compliance, 0.000 on ASR and benign success. The headline contrast is stable. We also record two harness bugs caught mid-experiment (a schema regex that blocked all deletes; a resume bug that silently duplicated seeds), each fixed and reported—the judge's judge applied to our own experiment.

7. Analysis: The Same Zero, Different Guarantees

The experiment's central image is two zeros. **V's 0.000 ASR is structural**: the confirmation gate reads platform-recorded events (the LLM cannot write them), and the sandbox enforces schema whitelists (arguments the attacker cannot produce are impossible). It holds while the model is maximally compromised (0.567 compliance) and across all 12 attack families, adaptive iteration, and white-box knowledge. **N's 0.000 ASR is behavioral**: the hardening prompt makes this particular model hedge—it complies with the injection by proposing the tool, then refuses the arguments. The same family's jailbreaks (our attacker was DeepSeek) cannot convert compliance into actionability. A different model, a stronger attacker, or a differently-phrased prompt could move N's zero; nothing can move V's within its ODD.

This is the deployment answer to "which defense should I buy?": **VAL selection buys a guarantee; intuition buys the model's current mood.** The cost difference is visible in the same table: both reach zero, but N's zero costs every benign action (0.000 benign success—users cannot transfer rent or send reports), while V's zero is lossless. Under VAL's usage criteria ([1], cost of silent error × encodability), the deployer's question becomes precise: *is the threat inside the anchor's ODD?* If yes, L2/L3 structure is available; if no, the honest answer is L2 correctness plus labeling, not a claim of safety.

8. Limitations

1. **Single model.** The agent and the adaptive attacker are DeepSeek-chat; cross-family attacker and victim combinations remain untested. D1's hedging wall is explicitly model-behavior-bound.
2. **Self-built testbed.** Scenarios, defenses, and the real-effect sandbox are ours; they are PPMF- and AgentDojo-inspired but not third-party. Attack success is counted on real state changes, but the tool surface is small.
3. **Attack strength.** Our 12 families include third-party AgentDojo payloads but not automated adaptive attackers (PAIR-grade) or optimization-based jailbreaks (GCG).
4. **LLM-rater classifications.** The level map is blind-rater-validated among same-family LLM raters ($\kappa \approx 0.8$); human-rater agreement is future work [1].
5. **Prediction sample.** 10/10 hits on a small, partially abstract-level sample; the mechanism (anchor → failure mode) is the claim, not the hit count.

9. Conclusion

The agent-security field has a deployment problem: too many defenses, no pre-purchase axis. We showed that VAL provides one. The anchor of a defense predicts how it fails—text rules fail to rewriting, model self-assessment fails to compliance, objective anchors fail outside their ODD, and structural anchors (confinement, information flow, data-control separation) hold by construction within it. We measured that choosing by this axis beats choosing by intuition on the same budget: identical security numbers, opposite guarantees, and a 100-point utility gap. And we located the L3 frontier in agent security: **confinement, not semantics**—the properties worth encoding into decidable systems are the ones that restrict what an agent can do, not the ones that judge whether it should.

> Others grade defenses by their claims. We grade them by their anchors—and the anchor decides.

References

1. Yin, Y. *Grading the Graders: Verification Autonomy Levels (L0–L5) for LLM Reasoning*. arXiv:2608.19009, 2026 (v2).
2. Xu, J., Xiao, Y., Shao, W., Liu, H., Li, X. *Memory Provenance Laundering in LLM Agents: A Non-Amplification Firewall for Persistent Memory*. arXiv:2607.29167, 2026.

3. Han, J., Buntine, W., Shareghi, E. *VerifiAgent: A Unified Verification Agent in Language Model Reasoning*. EMNLP 2025. arXiv:2504.00406.
4. Debenedetti, E., Zhang, J., Balunovic, M., Beurer-Kellner, L., Fischer, M., Tramer, F. *AgentDojo: A Dynamic Environment to Evaluate Attacks and Defenses for LLM Agents*. arXiv:2406.13352, 2024.
5. *Injection-Execution Dissociation: A Mechanistic Evaluation of Persistent Memory Attacks on Stateful LLM Agents*. arXiv:2605.08442, 2026.
6. *LLM-Based Test Oracles: Source-of-Authority Taxonomy—A Systematic Literature Review*. arXiv:2607.05031, 2026.
7. Lu, et al. *When Does Verification Pay Off? A Closer Look at LLMs as Solution Verifiers*. arXiv:2512.02304, 2025.
8. Zhou, A., et al. *Defending Against Alignment-Breaking Attacks via Robustly Aligned LLM (RA-LLM)*. arXiv:2309.14348, 2023.
9. Inan, H., et al. *Llama Guard: LLM-based Input-Output Safeguard for Human-AI Conversations*. arXiv:2312.06674, 2023.
10. Chen, X., et al. *CodeT: Code Generation with Generated Tests*. ICLR 2023.
11. Shi, T., et al. *Progent: Securing AI Agents with Privilege Control*. arXiv:2504.11703, 2025.
12. Costa, M., et al. *Securing AI Agents with Information-Flow Control*. arXiv:2505.23643, 2025.
13. Yin, Y. *Anchor Semantics Typology (intent/truth/effect)*. Project documentation, math_agent repository, 2026.
14. Wei, Q., et al. *A-MemGuard: A Proactive Defense Framework for LLM-Based Agent Memory*. arXiv:2510.02373, 2025.
15. Debenedetti, E., et al. *Defeating Prompt Injections by Design (CaMeL)*. arXiv:2503.18813, 2025.
16. Jain, N., et al. *Baseline Defenses for Adversarial Attacks Against Aligned Language Models*. arXiv:2309.00614, 2023.
17. Greshake, K., et al. *Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection*. arXiv:2302.12173, 2023.